PHASE 16 OF 18

Annotations & Reflection

Built-in and meta-annotations, retention and targets, writing custom annotations, processing them at runtime, the Reflection API (classes, fields, methods, constructors), dynamic proxies, costs and risks, and how Spring, JUnit and Appium use all of this

IN THIS PHASE

16.1 What Are Annotations?
16.2 Built-in Annotations
16.3 Meta-Annotations
16.4 Writing Custom Annotations
16.5 Processing Annotations at Runtime
16.6 The Reflection API
16.7 Costs and Risks of Reflection
16.8 Interview Questions
16.9 Quick Revision

16.1 What Are Annotations?

An **annotation** is **metadata**: a label attached to code (classes, methods, fields, parameters...) that doesn't change what the code does by itself. **Tools read it**: the compiler, build-time processors, or frameworks at runtime through reflection.

```
public String toString() { return "..."; }

void shouldTransferFunds() { }

private WebElement loginButton;

public class AccountController { }
```

Who reads annotations	When	Examples
Compiler	Compile time	`@Override`, `@FunctionalInterface`, `@SuppressWarnings`
Annotation processors	Compile time (generate code)	Lombok `@Data`, MapStruct, Dagger
Frameworks via reflection	Runtime	JUnit `@Test`, Spring `@Autowired`, JPA `@Entity`, Jackson `@JsonProperty`, TestNG `@DataProvider`

16.2 Built-in Annotations

Annotation	Purpose
`@Override`	Compile error if the method doesn't actually override or implement anything
`@Deprecated(since = "2.0", forRemoval = true)`	Marks an API as obsolete; callers get warnings
`@SuppressWarnings("unchecked")`	Silences specific compiler warnings (`"deprecation"`, `"rawtypes"`, `"unused"`...)
`@FunctionalInterface`	Compile error unless the interface has exactly one abstract method
`@SafeVarargs`	Promises a generic varargs method doesn't pollute the heap
`@Serial` (Java 14)	Marks serialization members (`serialVersionUID`, `writeObject`...) for compiler checks

```
public void legacyTransfer() { }

List<String> names = (List<String>) rawObject;     // suppress only where you're sure
```

16.3 Meta-Annotations

Annotations applied to **annotation definitions** to control how they behave.

@Retention: how long the annotation survives

RetentionPolicy	Kept in	Visible at runtime via reflection?	Example
SOURCE	Source code only (discarded by javac)	No	@Override, @SuppressWarnings, Lombok
CLASS (**default**)	The .class file	No	Bytecode tools
RUNTIME	The .class file **and** loaded into the JVM	**Yes**	@Test, @Autowired, @Entity, @FindBy

@Target: where it may be placed

ElementType	Allowed on
TYPE	Classes, interfaces, enums, records
FIELD	Fields (including enum constants)
METHOD	Methods
PARAMETER	Method parameters
CONSTRUCTOR	Constructors
LOCAL_VARIABLE	Local variables
ANNOTATION_TYPE	Other annotations (to build meta-annotations)
PACKAGE, MODULE, RECORD_COMPONENT	Packages, modules, record components
TYPE_USE, TYPE_PARAMETER (8)	Any use of a type: List<@NonNull String>

Other meta-annotation	Effect
@Documented	Include the annotation in Javadoc
@Inherited	A **class**-level annotation is inherited by subclasses
@Repeatable(Container.class) (8)	Allow the same annotation several times on one element

16.4 Writing Custom Annotations

```
import java.lang.annotation.*;

public @interface NotBlank {                // @interface declares an annotation type
    String message() default "must not be blank";
}

public @interface Range {
    long min() default 0;
    long max() default Long.MAX_VALUE;
}

public @interface Audited { }               // marker annotation: no elements

public @interface Smoke {
    String value();                         // element named 'value' -> @Smoke("login")
    int priority() default 3;
}
```

Rules for annotation elements

1. Elements look like methods with **no parameters** and **no** `throws`.
2. Allowed return types: primitives, `String`, `Class<?>`, enums, other annotations, and **arrays** of these. Not `Integer`, `List` or arbitrary objects.
3. `default` values must be **compile-time constants**; `null` is never allowed as a value.
4. If the only element you set is named `value`, you can omit the name: `@Smoke("login")`.
5. Array elements accept a single value without braces: `@SuppressWarnings("unused")` equals `@SuppressWarnings({"unused"})`.

16.5 Processing Annotations at Runtime

Reflection can read `RUNTIME` annotations. The examples below are tiny versions of what Bean Validation (`@NotBlank`, `@Size`) and JUnit do internally.

A mini validation framework

```
class Customer {
    @NotBlank(message = "name is required") private String name;
    @Range(min = 18, max = 120)                private int age;
    Customer(String name, int age) { this.name = name; this.age = age; }
}

static List<String> validate(Object obj) throws IllegalAccessException {
    List<String> errors = new ArrayList<>();
    for (Field f : obj.getClass().getDeclaredFields()) {
        f.setAccessible(true);                                  // read private fields
        Object value = f.get(obj);
        NotBlank nb = f.getAnnotation(NotBlank.class);
        if (nb != null && (value == null || value.toString().isBlank())) {
            errors.add(f.getName() + ": " + nb.message());
        }
        Range r = f.getAnnotation(Range.class);
        if (r != null) {
            long v = ((Number) value).longValue();
            if (v < r.min() || v > r.max())
                errors.add(f.getName() + ": must be between " + r.min() + " and " + r.max());
        }
    }
    return errors;
}

System.out.println(validate(new Customer(" ", 15)));
// [name: name is required, age: must be between 18 and 120]
System.out.println(Customer.class.isAnnotationPresent(Audited.class));   // true
```

A mini test runner (how JUnit/TestNG find tests)

```
class LoginTests {
    @Smoke(value = "valid login", priority = 1) public void validLogin()  { }
    @Smoke("locked account")
    public void lockedUser() { throw new IllegalStateException("locked"); }
    public void helper() { }                                            // not a test
}

Object suite = LoginTests.class.getDeclaredConstructor().newInstance();
for (Method m : LoginTests.class.getDeclaredMethods()) {
    Smoke s = m.getAnnotation(Smoke.class);
    if (s == null) continue;                                            // skip non-tests
    try {
        m.invoke(suite);
        System.out.println("PASS " + s.value());
    } catch (InvocationTargetException e) {                             // wraps the real failure
        System.out.println("FAIL " + s.value() + " -> " + e.getCause().getMessage());
    }
}
// PASS valid login
// FAIL locked account -> locked     (method order from reflection is NOT guaranteed)
```

> **NOTE**
> Compile-time **annotation processors** (`javax.annotation.processing`) read `SOURCE`/`CLASS` annotations during compilation and can **generate code**. That's how Lombok produces getters and setters, and how MapStruct generates mappers, with no runtime reflection cost.

16.6 The Reflection API

Reflection (`java.lang.reflect`) lets a program **inspect and use classes at runtime**: discover fields, methods, constructors and annotations, create objects, call methods and read or write fields, even ones it didn't know about at compile time.

Getting a Class object

```
Class<?> c1 = Customer.class;                       // class literal (compile-time known)
Class<?> c2 = new Customer("A", 30).getClass();     // from an instance (runtime type)
Class<?> c3 = Class.forName("com.bank.Customer");   // from a name (loads + initialises)
```

Inspecting a class

```
Class<?> c = ArrayList.class;
c.getName();                    // java.util.ArrayList
c.getSimpleName();              // ArrayList
c.getPackageName();             // java.util
c.getSuperclass();              // class java.util.AbstractList
c.getInterfaces();              // [List, RandomAccess, Cloneable, Serializable]
Modifier.isPublic(c.getModifiers());    // true
c.isInterface(); c.isEnum(); c.isRecord(); c.isArray();
```

Method	Returns
`getFields()` / `getMethods()` / `getConstructors()`	**Public** members, **including inherited** ones
`getDeclaredFields()` / `getDeclaredMethods()` / `getDeclaredConstructors()`	**All** members (any access) declared **in this class only**
`getField(name)` / `getDeclaredField(name)`	One field (`NoSuchFieldException` if missing)
`getMethod(name, paramTypes...)` / `getDeclaredMethod(...)`	One method
`getAnnotation(A.class)`, `isAnnotationPresent(A.class)`, `getAnnotations()`	Annotations

Creating objects, calling methods, accessing fields

```
class Account {
    private String accountNo;
    private double balance;
    public Account() { }
    public Account(String no) { this.accountNo = no; }
    private double applyFee(double amt) { return amt - 10; }
}

Class<Account> ac = Account.class;
Account a1 = ac.getDeclaredConstructor().newInstance();                  // no-arg
Account a2 = ac.getDeclaredConstructor(String.class).newInstance("SB-1"); // with args

Field bal = ac.getDeclaredField("balance");
bal.setAccessible(true);                          // bypass 'private' (same module/opened only)
bal.set(a2, 5000.0);
System.out.println(bal.get(a2));                  // 5000.0

Method fee = ac.getDeclaredMethod("applyFee", double.class);
fee.setAccessible(true);
System.out.println(fee.invoke(a2, 500.0));        // 490.0 (returns Object; boxed)
// Method.invoke(null, args) for static methods
```

Exception	When
`ClassNotFoundException`	`Class.forName` can't find the class

Exception	When
`NoSuchMethodException` / `NoSuchFieldException`	Wrong name or parameter types
`IllegalAccessException`	Accessing an inaccessible member without `setAccessible(true)`
`InvocationTargetException`	The **invoked method itself threw**: inspect `getCause()`
`InstantiationException`	Trying to instantiate an abstract class or interface
`InaccessibleObjectException` (Java 9+)	`setAccessible(true)` on a member of a module that doesn't `open` the package

Dynamic proxies: how Spring AOP and mocks work

`java.lang.reflect.Proxy` creates, **at runtime**, an object that implements given interfaces and routes every call to an `InvocationHandler`. This lets you add logging, timing, transactions, security or caching **without changing the real class**.

```
interface PaymentService { String pay(String acct, double amt); }

class RealPaymentService implements PaymentService {
    public String pay(String acct, double amt) { return "PAID " + amt + " to " + acct; }
}

PaymentService real = new RealPaymentService();
PaymentService logged = (PaymentService) Proxy.newProxyInstance(
        PaymentService.class.getClassLoader(),
        new Class<?>[]{PaymentService.class},
        (proxy, method, args) -> {                          // InvocationHandler
            System.out.println("-> " + method.getName() + Arrays.toString(args));
            long start = System.nanoTime();
            Object result = method.invoke(real, args);      // delegate to the real object
            long us = (System.nanoTime() - start) / 1000;
            System.out.println("<- " + result + " in " + us + "us");
            return result;
        });

logged.pay("SB-1001", 2500);
// -> pay[SB-1001, 2500.0]
// <- PAID 2500.0 to SB-1001 in 85us   (time varies)
```

> **NOTE**
> JDK proxies work only with **interfaces**. Spring uses them for interface-based beans and switches to CGLIB/ByteBuddy (subclass generation) for classes. That's why `@Transactional` doesn't apply when a method calls another method of the **same** object: the call bypasses the proxy.

16.7 Costs and Risks of Reflection

Concern	Detail
Performance	Reflective calls are slower than direct calls (lookups, boxing, access checks). Cache `Method`/`Field` objects; consider `MethodHandle`s for hot paths.
Breaks encapsulation	`setAccessible(true)` reaches private state and can violate class invariants
No compile-time safety	Typos in names fail only at **runtime** (`NoSuchMethodException`); refactoring tools may miss string names
Module restrictions	Since Java 9/16/17, deep reflection into JDK internals is blocked unless the module `opens` the package (`--add-opens` on the command line)
Security	Untrusted input deciding which class or method to call is dangerous (similar to deserialization gadgets)

> **TIP**
> Application code rarely needs reflection directly. Prefer interfaces, lambdas and polymorphism. Reflection belongs inside **frameworks and tools**: DI containers, test runners, serializers, ORMs and mocking libraries.

Where you meet annotations and reflection every day

Tool	Annotations	Reflection does...
JUnit 5	`@Test`, `@BeforeEach`, `@ParameterizedTest`, `@DisplayName`	Finds annotated methods, instantiates the test class, invokes the methods
TestNG	`@Test(groups, priority, dataProvider)`, `@DataProvider`	Same, plus grouping and ordering
Appium / Selenium PageFactory	`@FindBy`, `@AndroidFindBy`, `@iOSXCUITFindBy`	Creates element proxies and **injects** them into private fields
Spring	`@Component`, `@Autowired`, `@Value`, `@Transactional`	Scans classes, creates beans, injects dependencies, wraps them in proxies
Jackson	`@JsonProperty`, `@JsonIgnore`	Reads and writes fields and getters to map JSON
JPA / Hibernate	`@Entity`, `@Id`, `@Column`	Maps fields to columns and builds objects from rows
Mockito	`@Mock`, `@InjectMocks`	Generates mock subclasses and injects them into fields

16.8 Interview Questions

Q. What are annotations, and what are they used for?

Metadata attached to program elements. They are used by the compiler (`@Override`), by code generators at build time (Lombok), and by frameworks at runtime through reflection (`@Test`, `@Autowired`, `@Entity`).

Q. What are the retention policies?

`SOURCE` is discarded by the compiler. `CLASS` (the default) is stored in the `.class` file but not available at runtime. `RUNTIME` is available through reflection, which is needed for framework annotations.

Q. How do you create a custom annotation?

Declare it with `@interface`, add `@Retention` and `@Target`, and define elements as parameterless methods with optional `default` values (primitives, String, Class, enums, annotations or arrays of these). Read it with `getAnnotation()` at runtime.

Q. What is `@Inherited`?

A meta-annotation that makes a **class-level** annotation apply to subclasses too. It doesn't affect method or field annotations.

Q. What is reflection, and when is it used?

An API to inspect and manipulate classes, methods, fields and constructors at runtime. Frameworks use it for dependency injection, test discovery, serialization, ORM mapping, proxies and mocking.

Q. What is the difference between `getMethods()` and `getDeclaredMethods()`?

`getMethods()` returns all **public** methods including inherited ones. `getDeclaredMethods()` returns all methods of **any** access declared in that class only (not inherited).

Q. How can you access a private field via reflection? Is it always possible?

Use `getDeclaredField(name)`, then `setAccessible(true)`, then `get`/`set`. Since Java 9, this fails with `InaccessibleObjectException` when the target package belongs to a module that doesn't open it to your code.

Q. What is `InvocationTargetException`?

The wrapper exception thrown by `Method.invoke`/`Constructor.newInstance` when the invoked code itself throws. The real exception is available from `getCause()`.

Q. What is a dynamic proxy?

A runtime-generated class implementing given interfaces whose calls go to an `InvocationHandler`. It enables cross-cutting concerns (logging, transactions, security) without modifying the target. JDK proxies need interfaces.

Q. What are the drawbacks of reflection?

Slower calls, broken encapsulation, no compile-time checking, fragility under refactoring, module access restrictions, and security risks if inputs are untrusted.

16.9 Quick Revision

- Annotations = metadata read by the compiler, by processors (code generation) or by frameworks via reflection.
- Built-ins: `@Override`, `@Deprecated(since, forRemoval)`, `@SuppressWarnings`, `@FunctionalInterface`, `@SafeVarargs`, `@Serial`.
- Meta: `@Retention` (SOURCE / CLASS default / **RUNTIME**), `@Target` (TYPE, FIELD, METHOD, ...), `@Documented`, `@Inherited`, `@Repeatable`.
- Custom: `@interface`; elements are parameterless methods; types are primitives, String, Class, enum, annotation or arrays; defaults are constants; `value` shorthand.
- Read at runtime: `getAnnotation`, `isAnnotationPresent` (needs RUNTIME retention).
- Reflection: get a `Class` via `.class` / `getClass()` / `forName`. `getXxx` = public + inherited; `getDeclaredXxx` = all access, this class only.
- Create: `getDeclaredConstructor(...).newInstance(...)`. Call: `method.invoke(obj, args)`. Private: `setAccessible(true)` (subject to modules).
- `InvocationTargetException.getCause()` is the real error.
- `Proxy.newProxyInstance` + `InvocationHandler` = interface proxies (the basis of AOP and mocking).
- Reflection is powerful but slow and unsafe for app code; it belongs in frameworks.